5.1 PipeLine 模块 API 手册

1. 概述

本手册旨在指导开发人员使用 MicroPython 开发 AI Demo 时，构建完整的 Media 流程，实现从 Camera 获取图像和显示 AI 推理结果的功能。该模块封装了单摄双通道默认配置，一路将 Camera 的图像直接送给 Display 模块显示；另一路使用 `get_frame` 接口获取一帧图像供 AI 程序使用。

2. API 介绍

2.1 init

描述

PipeLine 构造函数，初始化 AI 程序获取图像的分辨率和显示相关的参数。

语法

```
from libs.PipeLine import PipeLine

pl=PipeLine(rgb888p_size=[1920,1080],display_size=
[1920,1080],display_mode='hdmi',debug_mode=0)
```

参数

参数名称	描述	输入 / 输出	说明
rgb888p_size	AI 程序的输入图像分辨率，list类型，包括宽高，如[1920,1080]	输入	默认[224,224]，根据 AI 程序决定
display_size	显示分辨率，list类型，包括宽高，如[1920,1080]	输入	默认[1920,1080]，根据显示屏幕决定
display_mode	显示模式，支持 `hdmi` 和 `lcd`，str类型	输入	默认 `lcd`,根据显示配置
debug_mode	调试计时模式，0计时，1不计时，int类型	输入	默认为0

返回值

返回值	描述
PipeLine	PipeLine实例

2.2 create

描述

PipeLine 初始化函数，初始化 Media 流程中的 Sensor/Display/OSD配置。

语法

```
pl.create()
```

参数

参数名称	描述	输入 / 输出	说明
sensor	Sensor实例	输入	可选，不同开发板有默认配置
hmirror	水平镜像参数	输入	可选，默认为 `None`,根据不同开发板默认配；设置时为bool类型，设为True或False
vflip	垂直翻转参数	输入	可选，默认为 `None`,根据不同开发板默认配置；设置时为bool类型，设为True或False
fps	sensor 帧率参数	输入	可选，默认60，设置Sensor的帧率

返回值

返回值	描述
无	

2.3 get_frame

描述

获取一帧图像给 AI 程序使用，获取图像分辨率为PipeLine构造函数设置的rgb888p_size，图像格式为Sensor.RGBP888，返回时转换成ulab.numpy.ndarray格式。

语法

```
img=pl.get_frame()
```

返回值

返回值	描述
img	格式为ulab.numpy.ndarray，分辨率为rgb888p_size的图像数据

2.4 show_image

描述

将在 `pl.osd_img` 上绘制的 AI 结果在 Display 上叠加显示。`pl.osd_img` 是 `create` 接口初始化的一帧格式为 `image.ARGB8888` 的空白图像，用于绘制 AI 结果。

语法

```
pl.show_image()
```

返回值

返回值	描述
无	

2.5 destroy

描述

反初始化PipeLine实例。

语法

```
img=pl.destroy()
```

返回值

返回值	描述
无	

3. 示例程序

以下为示例程序:

```
from libs.PipeLine import PipeLine, ScopedTiming
from media.media import *
import gc
import sys,os

if __name__ == "__main__":
    # 显示模式，默认"hdmi",可以选择"hdmi"和"lcd"
    display_mode="hdmi"
    if display_mode=="hdmi":
        display_size=[1920,1080]
    else:
        display_size=[800,480]

    # 初始化PipeLine，用于图像处理流程
    pl = PipeLine(rgb888p_size=[1920,1080], display_size=display_size,
display_mode=display_mode)
    pl.create()  # 创建PipeLine实例
    try:
```

```
        while True:
            os.exitpoint()                          # 检查是否有退出信号
            with ScopedTiming("total",1):
                img = pl.get_frame()                # 获取当前帧数据
                print(img.shape)
                gc.collect()                        # 垃圾回收
    except Exception as e:
        sys.print_exception(e)                      # 打印异常信息
    finally:
        pl.destroy()                                # 销毁PipeLine实例
```

上述代码中，通过`pl.get_frame()`接口获取一帧分辨率为rgb888p_size的图像，类型为ulab.numpy.ndarray，排布为CHW。基于这段代码，您可以专注于 AI 推理部分的操作。

> **计时工具**ScopedTiming
>
> ScopedTiming 类在PipeLine.py模块内，是一个用来测量代码块执行时间的上下文管理器。上下文管理器通过定义包含 `__enter__` 和 `__exit__` 方法的类来创建。当在 with 语句中使用该类的实例，`__enter__` 在进入 with 块时被调用，`__exit__` 在离开时被调用。
>
> ```
> from libs.PipeLine import ScopedTiming
>
> def test_time():
> with ScopedTiming("test",1):
>     #####代码#####
>     # ...
>     ###############
> ```